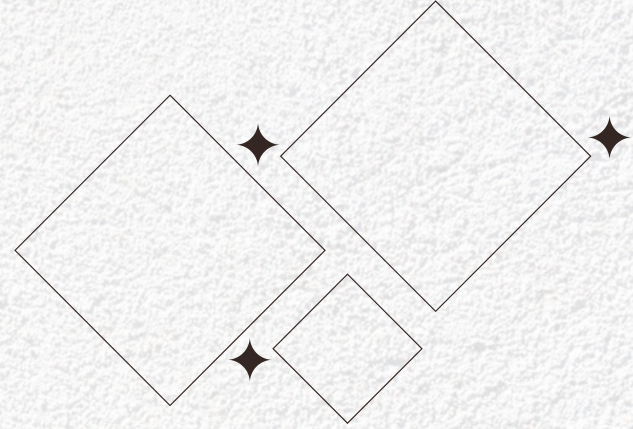
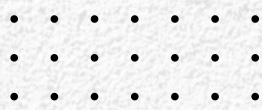


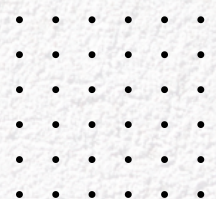
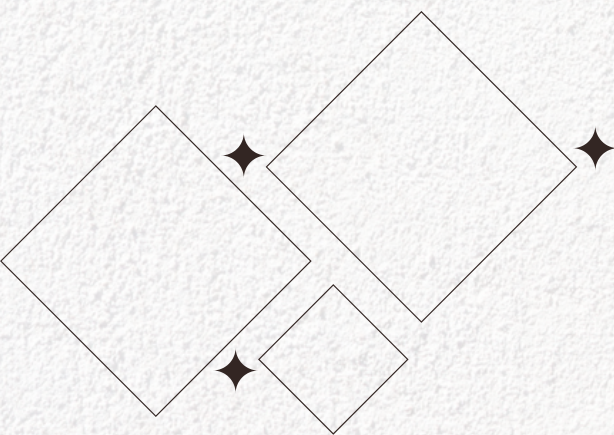


Laboratorio 8- traversal sequences blocked with absolute path bypass

CNO IV Programación WAP

Maestro: Servando López Contreras

Lopez Alcocer Monica Isabel - 179827



Índice

Índice	1
1. Ficha técnica del laboratorio.....	2
2. Objetivo del laboratorio.....	2
3. Marco teórico y fundamentación técnica	2
3.1 ¿Qué es el File Path Traversal?	2
3.2 El problema de las defensas por lista negra (blacklist)	2
3.3 Ruta relativa vs. ruta absoluta (núcleo de este laboratorio)	2
3.4 Variantes de evasión de filtros	3
3.5 Impacto sobre la tríada CIA.....	3
3.6 Clasificación OWASP / CWE	3
4. Explicación técnica de la vulnerabilidad (caso específico).....	3
5. Procedimiento paso a paso.....	4
6. Evidencias — Capturas de pantalla obligatorias	4
6.1 Request interceptado.....	4
6.2 Payload utilizado	4
6.3 Confirmación «Lab Solved»	5
7. Explicación del payload.....	5
8. Mitigación recomendada	6
9. Conclusión	6

1. Ficha técnica del laboratorio

Nombre	File path traversal, traversal sequences blocked with absolute path bypass
Plataforma	PortSwigger — Web Security Academy
URL	https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass
Nivel	Apprentice
Tipo de explotación	File Path Traversal — evasión de filtro mediante ruta absoluta
Clasificación	CWE-22 · OWASP Top 10 2021 → A01: Broken Access Control
Herramienta	Burp Suite Community Edition (Proxy + Repeater)
Objetivo de resolución	Leer el archivo /etc/passwd del servidor

2. Objetivo del laboratorio

Igual que en el caso simple, la tienda carga la imagen de cada producto mediante un parámetro filename. La diferencia es que **esta aplicación sí implementa una defensa: bloquea las secuencias de recorrido** `../`. El objetivo sigue siendo leer `/etc/passwd`, pero para lograrlo hay que **evadir ese filtro. La debilidad es que la aplicación toma el nombre del archivo como relativo a un directorio de trabajo predeterminado y no impide el uso de rutas absolutas**, de modo que una ruta que empiece desde la raíz del sistema (`«/etc/passwd»`) escapa del directorio sin usar ninguna secuencia `../`.

3. Marco teórico y fundamentación técnica

3.1 ¿Qué es el File Path Traversal?

El **File Path Traversal** (o *Directory Traversal*) es una vulnerabilidad de control de acceso que permite leer —y a veces escribir— archivos fuera del directorio previsto por la aplicación, cuando ésta construye una ruta del sistema de archivos a partir de entrada del usuario **sin validarla ni canonicalizarla**. El vector clásico son las secuencias `../` para «subir» de directorio; sin embargo, cuando la aplicación intenta bloquear ese vector con filtros incompletos, aparecen técnicas de evasión como la de este laboratorio.

3.2 El problema de las defensas por lista negra (blacklist)

Una defensa frecuente —pero insuficiente— consiste en **buscar y bloquear la cadena** `..` o `../` en la entrada (enfoque de *lista negra*). El problema es que una lista negra solo detiene los patrones que su autor anticipó. La lectura arbitraria de archivos **no requiere obligatoriamente secuencias de recorrido**: si la aplicación también acepta **rutas absolutas**, el atacante puede referenciar el archivo objetivo directamente desde la raíz del sistema, sin escribir un solo `..`, evadiendo así por completo el filtro.

3.3 Ruta relativa vs. ruta absoluta (núcleo de este laboratorio)

- **Ruta relativa:** se interpreta a partir de un directorio de trabajo base. Ej.: 25.jpg → /var/www/images/25.jpg. Para escapar de ese directorio se necesitan secuencias ../ (que aquí están bloqueadas).
- **Ruta absoluta:** comienza con / y se interpreta desde la raíz del sistema, **ignorando el directorio de trabajo**. Ej.: /etc/passwd se resuelve directamente como /etc/passwd, sin necesidad de subir niveles.

Cuando el código de la aplicación entrega el filename al sistema de archivos y éste recibe una ruta que ya empieza por /, la ruta base se descarta y se abre el archivo absoluto. Por eso, **bloquear ../ pero permitir rutas absolutas deja la puerta abierta**.

3.4 Variantes de evasión de filtros

Según cómo esté implementado el filtro, existen distintas técnicas para evadirlo. La de este laboratorio es la primera de la tabla:

Defensa del servidor	Técnica de evasión
../ bloqueado / eliminado, permite absolutas	/etc/passwd (ruta absoluta) ← este lab
Elimina ../ una sola vez (no recursivo)	../../../../../../../../etc/passwd
Filtra ../ por cadena literal	..%2f o doble codificación ..%252f
Valida que empiece por una carpeta esperada	/var/www/images/../../../../etc/passwd
Valida que termine en extensión (.png)	../../../../etc/passwd + byte nulo %00.png

3.5 Impacto sobre la tríada CIA

- **Confidencialidad (alto):** lectura de archivos sensibles: credenciales, código fuente, configuraciones con contraseñas de BD, claves SSH, tokens.
- **Integridad (condicional):** si la función permite escritura, el mismo defecto puede usarse para sobrescribir archivos y llegar a ejecución de código.
- **Disponibilidad (indirecto):** la información expuesta suele ser el punto de partida de ataques que sí afectan la disponibilidad del servicio.

3.6 Clasificación OWASP / CWE

- **CWE-22:** Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').
- **OWASP Top 10 2021 — A01: Broken Access Control.**

4. Explicación técnica de la vulnerabilidad (caso específico)

Al abrir el detalle de un producto («View details»), el navegador solicita la imagen mediante el endpoint /image, visible en el HTTP history y en el Site map de Burp. La petición tiene la forma:

```
GET /product?productId=14.jpg HTTP/2
Host: <LAB-ID>.web-security-academy.net
```

La aplicación aplica un filtro que **bloquea las secuencias de recorrido ../**, por lo que un intento como filename=../../../../etc/passwd es rechazado. Sin embargo, el filtro **no contempla las rutas absolutas** y la aplicación interpreta el valor de filename como relativo a un directorio de trabajo predeterminado,

entregándolo al sistema de archivos sin canonicalizar. Como consecuencia, al enviar una ruta que empieza en la raíz («/etc/passwd»), el sistema operativo la abre directamente, ignora el directorio base y devuelve el contenido del archivo. La defensa por lista negra queda así evadida sin usar un solo ...

5. Procedimiento paso a paso

1. Iniciar el laboratorio con «Access the lab» y abrir la tienda desde el navegador de Burp (Proxy → «Open Browser») para que el tráfico pase por el proxy.
2. Reconocimiento. Abrir un producto con «View details» y, en Proxy → HTTP history, localizar la petición que carga la imagen: GET /product?productId=....
3. Enviar a Repeater. Clic derecho sobre esa petición /image → «Send to Repeater» (Ctrl+R).
4. Aplicar el bypass con ruta absoluta. Sustituir el valor por la ruta absoluta del archivo por:

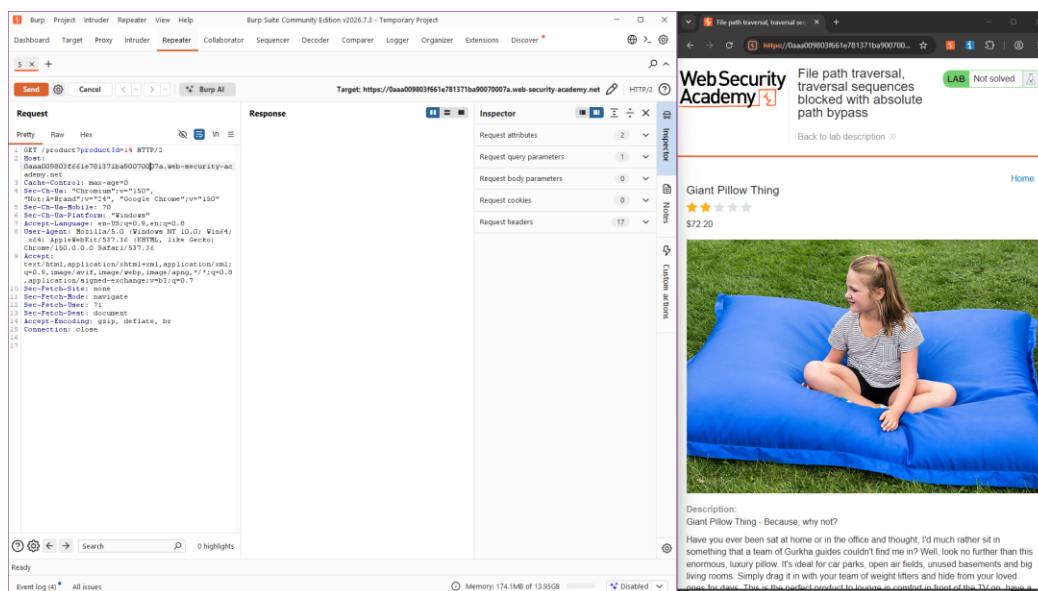
```
GET /image?filename=/etc/passwd HTTP/2
```

5. Enviar la petición con «Send» y revisar el panel Response.
6. Confirmar la explotación. La respuesta es HTTP/2 200 OK con Content-Type: image/jpeg y, en el cuerpo, el contenido de /etc/passwd (líneas como root:x:0:0:root:/root:/bin/bash). Queda demostrada la lectura arbitraria de archivos evadiendo el filtro.
7. Verificar «Lab Solved». El banner superior cambia de «Not solved» a «Solved»

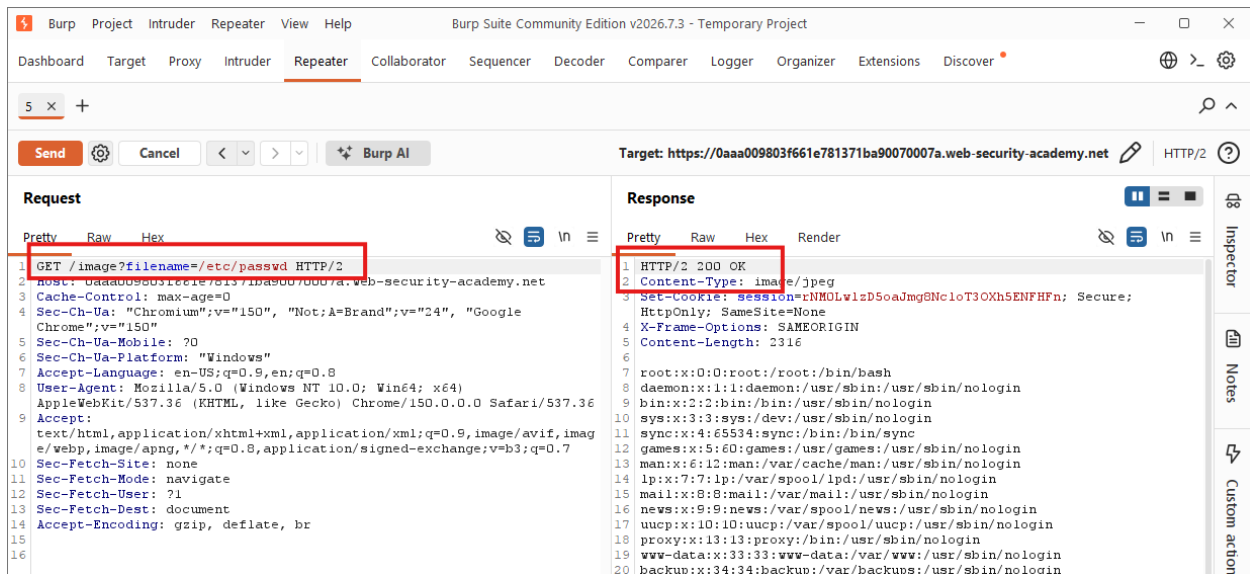
6. Evidencias — Capturas de pantalla obligatorias

Evidencias del laboratorio

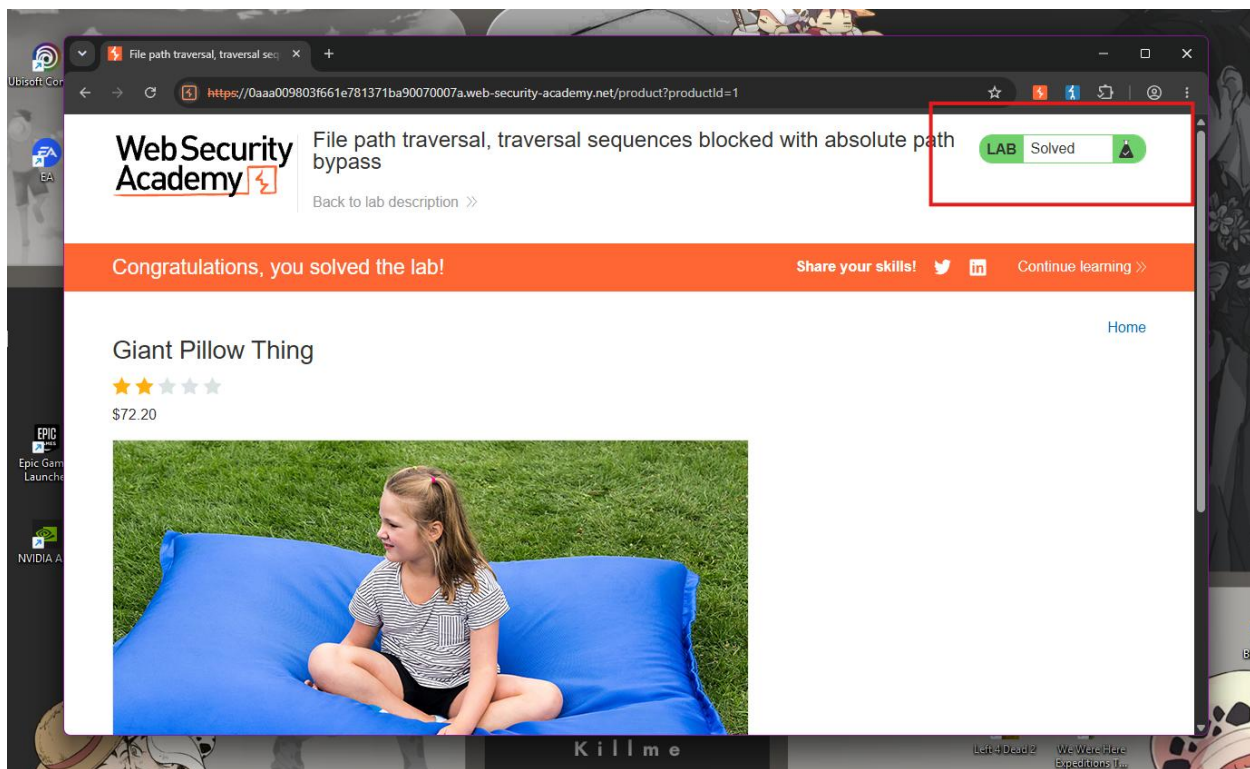
6.1 Request interceptado



6.2 Payload utilizado



6.3 Confirmación «Lab Solved»



7. Explicación del payload

/etc/passwd

Elemento	Función / razón de que funcione
/ inicial	Marca la ruta como ABSOLUTA. El sistema de archivos la interpreta desde la raíz del sistema e ignora el directorio de trabajo base de la aplicación.

Elemento	Función / razón de que funcione
etc/passwd	Ruta estándar del archivo objetivo en Linux. Es legible por cualquier usuario, por lo que sirve como prueba de concepto sin exponer secretos críticos.
Ausencia de ../	Al no contener secuencias de recorrido, el payload evade por completo el filtro que solo busca y bloquea ../.

Contraste con el caso simple: allí el payload era `../../../../etc/passwd` porque no había filtro. Aquí ese payload **es bloqueado**; la clave del bypass es que `/etc/passwd` logra el mismo objetivo (leer el archivo) **sin usar recorrido relativo**, aprovechando que la aplicación acepta rutas absolutas y no verifica que el archivo resuelto permanezca dentro del directorio de imágenes.

¿Por qué /etc/passwd? Está presente en cualquier sistema Linux, tiene permisos de lectura globales y un formato reconocible (`usuario:x:UID:GID:...:/home:/shell`), por lo que su lectura confirma la vulnerabilidad de forma inequívoca y sin causar daño.

8. Mitigación recomendada

Este laboratorio demuestra por qué una lista negra no basta. La defensa debe ser positiva (lista blanca) y verificar la ruta ya resuelta:

- **No usar listas negras como único control:** bloquear solo `../` es evadible con rutas absolutas, codificación URL, doble codificación o secuencias anidadas.
- **Rechazar rutas absolutas:** descartar cualquier valor que comience por `/` (o `\`, o que contenga `:` en Windows).
- **Validación por lista blanca:** aceptar únicamente nombres que cumplan un patrón estricto, p. ej. `^[a-zA-Z0-9_-]+\.(png|jpg)$`, y rechazar todo lo demás.
- **Canonicalizar y verificar el contenedor:** resolver la ruta final (`realpath / getCanonicalPath`) y comprobar que sigue dentro del directorio base permitido. **Este control detiene tanto .. como las rutas absolutas.**
- **Referencias indirectas:** exponer un identificador (`imageId=14`) que el servidor mapee internamente a la ruta física mediante una tabla, en lugar del nombre de archivo real.
- **Mínimo privilegio:** ejecutar el servicio con un usuario sin acceso de lectura a archivos sensibles del sistema, para limitar el impacto aun si el control falla.

9. Conclusión

El laboratorio evidencia que una defensa incompleta puede dar una falsa sensación de seguridad: aunque la aplicación bloquea las secuencias `../`, la lectura arbitraria de archivos se logró igualmente enviando la ruta absoluta `/etc/passwd`, que evade el filtro por no contener recorrido relativo. Se recuperó `/etc/passwd` con una respuesta `200 OK`, resolviendo el laboratorio. La lección práctica es clara: las listas negras son frágiles; el control correcto es la validación por lista blanca junto con la canonicalización y la verificación de que el archivo resuelto permanezca dentro del directorio permitido.